EECE7352 Computer Architecture

Homework 3

Sakthivel P. Sivakumar

NUID: 002312294

Part B: (30 Points)

Your assignment is to write a simulator to model three simple conditional branch predictors. The first predictor will be indexed using only the branch instruction address and will have 64 1-bit predictors, saving the previous action of the last branch to predict the outcome of the next iteration of the branch. The second predictor will also use only the branch instruction address to index into the predictor and will have 32 2-bit counters to predict branches. You should decide how to use the 2-bit counters to predict branches, and how to initialize the counters before simulation starts. The third predictor will include a pattern history register, recording the outcome (taken/not taken) of the last N branches (part of the assignment is to find the best value for N). You will XOR the pattern history with the branch instruction address to form your index. You have 32 2-bit counters to predict branches. Again, you decide how to use the 2-bit counter to predict branches, and how to initialize the counters before simulation starts. For these predictors, you will not use a tag, so you may have multiple branches aliasing into the same table entry. This is intended.

Ans:

In this part I have presented the design and performance evaluation of three branch prediction schemes tested on a branch trace containing 16,416,279 branch instructions. The predictors were evaluated based on their prediction accuracy, with Predictor 2 (2-bit bimodal) achieving the highest accuracy at 70.315%.

Predictor -1 (1-bit, 64 entries) [predictor1.py]

```
PS D:\M.S. in ECE\Fall 2025\EECE_7352_Computer_Architecture\Homeworks\HW_3> python predictor1.py itrace.out.gz
Predictor 1: 1-bit, 64 entries
================================================
Total branches      : 16,416,279
Correct predictions : 11,408,039
Incorrect predictions: 5,008,240
Accuracy            : 69.492%
```

For this branch predictor-1, the design features a table with 64 entries, where each entry is a **single 1-bit predictor**. Indexing into this table is handled directly using the branch's PC address; specifically, the index is calculated using the lower 6 bits of the PC, corresponding to PC & 63. The prediction logic is straightforward: if the bit is 1, the branch is predicted as Taken, and if it is 0, it is predicted as Not-Taken. All entries are initialized to 0, so the predictor will default to "predict not-taken" for all branches initially. The update policy is to set the entry's bit to the actual outcome of the branch (1 for taken or 0 for not-taken).

Predictor -2 (2-bit, 32 entries) [predictor2.py]

```
PS D:\M.S. in ECE\Fall 2025\EECE_7352_Computer_Architecture\Homeworks\HW_3> python predictor2.py itrace.out.gz
Predictor 2: 2-bit, 32 entries
================================================
Total branches      : 16,416,279
Correct predictions : 11,543,027
Incorrect predictions: 4,873,252
Accuracy            : 70.315%
```

For this branch predictor-2, the design uses a table of 32 entries, with each entry being a **2-bit saturating counter that represents four states: Strongly Not-Taken (00), Weakly Not-Taken (01), Weakly Taken (10), and Strongly Taken (11)**. Indexing into the table is handled directly, using an index calculated from the lower 5 bits of the branch's PC address (PC & 31). All entries are initialized to the 01 state (Weakly Not-Taken). The prediction logic is to predict TAKEN if the counter is in state 10 or 11, and predict NOT-TAKEN if it is in state 00 or 01. After the branch resolves, the counter is updated: it is incremented (saturating at 11) if the branch was taken and decremented (saturating at 00) if it was not-taken. This 2-bit counter provides hysteresis, meaning it requires two consecutive mispredictions in the same direction to flip the prediction from taken to not-taken (or vice versa), making it more resilient to occasional, non-typical branch outcomes and more effective for loops.

Predictor – 3 (32 Entries with Pattern History Register) [predictor3.py)

```
Predictor 3: GShare, 32 entries, N=8
================================================
Total branches      : 16,416,279
Correct predictions : 10,990,279
Incorrect predictions: 5,426,000
Accuracy            : 66.947%
```

In this predictor-3 (pattern history register) design specifies a correlating branch predictor featuring a 32-entry counter table, where each entry is a 2-bit saturating counter. It also employs an **N-bit Pattern History Register** (PHR) that tracks the outcomes of the last N branches. The key to this design is its indexing method, which XORs the branch PC with the PHR and uses the lower 5 bits of the result ((PC ^ PHR) & 31) as the index. This correlation allows branches with similar history patterns to share predictor entries, capturing global control flow patterns. For initialization, the counter table entries are all set to state 01 (Weakly Not-Taken), and the PHR is initialized to 0. The prediction and update logic uses the same 2-bit saturating counter mechanism as the previous predictor. After a branch resolves, the PHR is updated by being left-shifted by 1 bit and having the branch outcome (1 for taken, 0 for not-taken) inserted at the LSB, which is then masked to N bits. The total hardware storage required is 64 bits for the counter table plus N bits for the PHR, and it operates as a direct-mapped table without tags.

Overall Performance Comparison:

Based on a **trace of 16,416,279** total branches, Predictor 1, the simple 1-bit predictor with 64 entries, achieved an accuracy of **69.492%**, resulting in a misprediction rate of 30.508%. Predictor 2, which used 2-bit saturating counters in a 32-entry table, was the best-performing predictor. It achieved an accuracy of **70.315%** with a misprediction rate of 29.685%, correctly predicting 11,543,013 branches. For Predictor 3 (GShare), testing was conducted with **N values from 1 to 16**. The best N value was found to be N=1, which yielded an accuracy of **68.962%.** Other values, such as N=5, N=8, and N=10, resulted in lower accuracies around 67%,

likely due to increased table aliasing. The final performance for the best GShare predictor (N=1) had a misprediction rate of 31.038%.

Why Predictor - 2 performed well?

Predictor 2's superior performance is driven by several key factors. The most significant is the **hysteresis** provided by its 2-bit saturating counter, which offers resistance to single-outcome changes. While a 1-bit predictor flips its prediction immediately on any misprediction, the 2-bit counter **requires two consecutive opposite outcomes** to do so, making it more stable. For example, in a loop with an occasional anomaly (e.g., one 'Not-Taken' in a string of 'Taken' branches), the 1-bit predictor would mispredict twice—once for the anomaly and again when the pattern returns. The 2-bit predictor, however, would only mispredict on the anomaly and would not change its overall prediction, thus handling the temporary change more effectively.

This hysteresis is possible because the 2-bit predictor has a better state representation, using four states (Strongly/Weakly Taken/Not-Taken) instead of just two. This allows for gradual transitions and captures the "confidence level" of a branch's bias, unlike the 1-bit predictor, which cannot distinguish between a strong or weak bias.

Extra Credit (improved prediction accuracy using Perceptron)

The **Perceptron** predictor is a **neural network-based branch prediction** scheme that uses machine learning to predict branch outcomes. Unlike simple counter-based predictors, perceptrons can learn complex, non-linear correlations between branch history and outcomes.

Predictor- 4 (Perceptron Predictor)

```
PS D:\M.S. in ECE\Fall 2025\EECE_7352_Computer_Architecture\Homeworks\HW_3> python
 predictor_perceptron.py itrace.out.gz --sweep
Sweeping history length from 4 to 16...
============================================================
Testing H=4... Accuracy=76.104%
Testing H=5... Accuracy=76.733%
Testing H=6... Accuracy=77.425%
Testing H=7... Accuracy=77.781%
Testing H=8... Accuracy=78.348%
Testing H=9... Accuracy=78.540%
Testing H=10... Accuracy=78.884%
Testing H=11... Accuracy=79.273%
Testing H=12... Accuracy=79.676%
Testing H=13... Accuracy=80.058%
Testing H=13... Accuracy=80.058%
Testing H=14... Accuracy=80.412%
Testing H=15... Accuracy=80.577%
Testing H=16... Accuracy=80.902%
============================================================
Best history length: H=16 with accuracy=80.902%
```

The perceptron branch predictor design specifies an architecture consisting of a 32-entry table. Indexing into this table is done using direct PC indexing based on the lower 5 bits (PC & 31). **Each entry in the table contains one perceptron with (H+1) weights**, which includes 1 bias weight and H history weights, one for each bit of history. The predictor also maintains a global history as an H-bit register, which stores +1 for taken branches and -1 for not-taken branches. The key parameters for this design include a history length (H) of 8 bits. The training threshold (θ) is set according to the formula $\theta = (1.93 \times H + 14)$, which results in a threshold of 29 for the 8-bit history length.

Advantages of Perceptron over 2- bit counters

- It can learn complex, non-linear patterns in branch behaviour (like XOR patterns) that 2-bit counters cannot.
- It learns multiple correlations by having a specific weight for each position in the branch history, unlike a 2-bit counter which only tracks "recent behaviour."
- It features adaptive learning, allowing it to continue training and refining its weights even on correct predictions that had low confidence.
- It has a vast pattern capacity, giving it the ability to distinguish up to 2^H unique patterns and capture more nuanced relationships.

Conclusion

the perceptron predictor represents a significant leap in branch prediction sophistication. In fact, **modern processors often use advanced variants of the perceptron**, such as TAGE-SC or PPM. For this specific 32-entry constraint, a perceptron with a history length (H) between 8 and 12 should theoretically provide **5-10% better accuracy** than the standard 2-bit counters. However, this improved accuracy comes at a significant cost, **requiring approximately 30-40 times more storage to implement**.